

架构讨论：Harness + 插件化（万物皆 Skill）

> 2026-09-25 | 提出人：用户 | 本文档回答三件事：你的方案解决了什么、哪些地方必须补硬、还有哪些要敲定

> 配套图：[docs/platform-vision/diagrams/harness-architecture.html](#)（架构图 + 流程图）

0. 一页纸结论

问题	结论
方向对不对	对，而且是升级。 原来的编排（意图判定→工具计划→证据装配→答案成型） 写死在代码里 ；改成" harness 底座 + 万物皆 skill"后， 加一种能力 = 加一个 skill ，不用改编排内核
最大的变化	从「规则优先、LLM 兜底」改成「 LLM/决策模型为主 」。这是一次 根本性立场切换 ，收益是能力上限高，代价是 不确定性变高 （会编、会漂、可能幻觉出 <code>truncate table</code> ）
因此必须补硬的三处	① 事实不得由模型生成 ——要用 机制 保证（执行器回执校验），不能靠提示词；② 防火墙 fail-closed （拿不准就拒），且中高风险要 留痕 + 复核 ；③ 知识回写也是一种写操作 ，必须过审、有版本、可回滚，否则知识库会被幻觉污染
原设计里缺的一层	智能防火墙 （你提的）。我原方案只有"写操作必经生成→审核→执行"的原则，没有独立的策略引擎——这是 真正的缺口 ，补上后整条链路才闭环
可复用的已有资产	知识库（75 口径 / 298 字段 / 139 术语 / 76 规则）、SQL 血缘解析、生成四层（SQL→链路→落地→反向校验）、海豚插件（执行器雏形）、继承下来的编排骨架（可改造成 skill 注册表）

1. 组件级对照（你的定义 ↔ 现状 ↔ 差距）

你的组件	职责	我们已有的	差距 / 要做的事
Harness 底座	会话与上下文、决策（LLM/决策模型）、技能注册与调度、结果回传	现有编排（entity/intent/planner/assemble/answer） + 前端对话工作台 + 会话审计落库	需要重构成 skill 注册表 + 决策循环 ；决策过程与工具调用要可观测、可回放
Skill（技能/Agent）	一类能力一个 skill：解析、生成、查询、调度、格式化...	现有工具方法（analyze/upstream/impact/search/metric/report）	需要 skill 契约 （schema/权限/幂等/超时/失败语义），并把现有工具包装成 skill
知识库	业务口径 + 表 + 表字段 + 术语 + 规则	☒ 已存在且实测可用（ <code>/kb/search</code> 、 <code>/kb/metric</code> 、 <code>/kb/summary</code> ）	需要 写入通道 + 审核 + 版本 + 回滚 （现在只有读和离线构建）

智能防火墙	权限校验 + 风险判定，防幻觉指令影响生产	✗ 没有（只有静态校验 <code>/generate/validate</code> ）	新建：策略表 + 动作分级 + 审批令牌 + 全量留痕
执行层（执行器）	对接调度工具、大数据组件监控；新增一个调度器只加一个适配器	海豚插件（旁路集成，已验证）；监控：无	抽象成 调度适配器协议 （提交/暂停/重跑/补数/查状态）+ 监控适配器协议 （集群/库/表状态）
前端（Web 为主）	harness 的交互面；结果按类型渲染	✅ 对话工作台 + 证据面板（口径/血缘/审计）已跑通	结果渲染要变成 可插拔视图 （表格、依赖图、SQL 高亮、执行计划、监控卡片）
初始化批量解析（可选）	一次性把调度器里已有任务的业务知识抽出来	有工作流级解析能力（ <code>/analyze-workflow</code> ）+ 离线导入思路	需要 批量任务编排 + 幂等 + 进度 + 人工抽检 ，且默认 只读不落库 （先出报告）

2. 必须补硬的三处（按风险排序）

2.1 事实不得由模型生成 —— 要用机制，不能靠提示词

LLM 为主之后，最大的风险不是“答得不好”，而是**答得很像真的**。原设计有一条铁律“事实来自内核”，那是在规则模式下天然成立的；换成 LLM 主导后，必须换成**机制**：

- **执行器回执校验**：skill 返回必须带结构化回执（`source/endpoint/ms/rows/affected`），harness 只允许用回执里的数字与标识符组答案；模型输出里出现回执之外的**表名/数字/字段名**，一律标红或打回重写。
- **引用强制**：每条结论必须挂 `evidence[]`（证据 id），无证据的结论**不允许**输出（沿用现有 `Answer` 模型校验，铁律 1 继续生效）。
- **敏感动作白名单**：模型只能“提出”动作，不能“决定”动作——动作由防火墙裁决（见 2.2）。

2.2 智能防火墙：fail-closed、分级、留痕、可复核

建议做成**独立组件**（而不是散在 skill 里的 if-else），判定顺序为：**身份 → 权限 → 风险分级 → 处置**。

级别	例子	处置
L0 只读	查血缘、检索口径、看集群状态、 <code>select</code>	直接通过（仍留痕）
L1 低风险写	生成 SQL 草稿、生成调度流程草稿、写临时表	通过 + 强制附“未执行”标注
L2 中风险	建表、改表结构、上线调度流程、重跑历史实例	回给用户确认 （展示影响面：下游 N 张表 / M 个任务）
L3 高风险	补数超大范围、删除分区、修改生产口径、批量重跑	双人复核 （提交人 + 负责人），带时效令牌
L4 禁止	<code>drop table</code> 、 <code>truncate</code> 、无 <code>where</code> 的 <code>delete/update</code> 、跨域导出	直接拒绝 并解释原因，记录尝试

要点：① **拿不准按上一级处理**（fail-closed）；② 每次判定都要落审计（谁、什么动作、判定依据、结果）；③ 审批令牌**一次性、有时效、绑定具体动作**，不能复用。

2.3 知识回写 = 写操作，同样要过审

"有价值的知识，用户确认后落盘"这个闭环很好，但**知识库是下游所有回答的地基**——被幻觉污染一次，后面所有回答都会错。建议：

- 回写走**候选池**（`candidate` 状态），不直接进正式口径；
- 需要**责任人审核**（谁的表谁审）+ **去重/冲突检测**（同字段两条口径公式不同 → 必须仲裁，不允许并存）；
- **版本化 + 可回滚**（每条口径有版本号与来源脚本，能一键回退）；
- 回写内容**必须带来源证据**（脚本 + 行号 + 摘要），无来源不允许入库。

3. Skill 契约（最小字段集，供敲定）

一个 skill 想被 harness 调用，至少要声明：

字段	说明	例子
<code>name / version</code>	唯一名 + 版本	<code>lineage.analyze@1</code>
<code>when</code>	什么时候用（给决策模型看的描述）	"用户给了 SQL 或问某脚本的血缘"
<code>input_schema / output_schema</code>	入参出参 JSON Schema（供模型填参数 + 供校验）	<code>{sql, dialect, depth}</code>
<code>side_effect</code>	<code>none / read / write / ddl / dml</code>	决定防火墙最低级别
<code>auth_scope</code>	需要的权限点	<code>lineage:read</code> 、 <code>schedule:write</code>
<code>idempotent / timeout / retry</code>	幂等性、超时、重试语义	幂等 + 30s + 仅连接类错误重试
<code>evidence_kind</code>	返回值能作为哪类证据	<code>lineage / metric / report / plan</code>
<code>renderer</code>	结果用哪个视图渲染	<code>table / graph / sql / diff</code>
<code>budget</code>	单次会话内的调用上限（防循环）	最多 6 次

调度可控性是这类架构最容易踩的坑：必须限制「单轮最大 skill 数」「同 skill 最大重试」「总 token/时间预算」，否则模型会绕圈。

4. 执行器协议（建议的最小方法集）

```
SchedulerAdapter（调度适配器，一个平台一个实现）
list_workflows(project) / get_workflow(id)
create_draft(pipeline) / validate(draft)           # 只产出草稿，不落地
release(id) / offline(id)                         # L2 起，需审批令牌
run(id, params) / rerun(instance_id)              # L2/L3
get_instance_status(id) / fetch_logs(instance_id)
list_instances(workflow, timerange)                # 补数范围评估用
```

```
MonitorAdapter (监控适配器)
cluster_health() / queue_status()
db_status(db) / table_stats(table) / slow_queries(top_n)
lineage_graph(scope) # 已有: 血缘图谱
```

要点: 新增一个调度平台 = 新增一个适配器, 上游 (harness/防火墙/skill) 零改动——这正是你说的解耦, 落点就在这两个协议上。

5. 主流程 (与图对应)

用户指令

→ Harness (会话上下文 + 决策模型: 意图 → 选 skill → 组参)

→ 【智能防火墙】身份/权限/风险分级

- └─ L0/L1 → 执行器 → 回执 → 模型汇总 → 前端渲染 (按 renderer)
- └─ L2 → 回用户确认 (附影响面) → 带令牌 → 执行器 → 回执 → ...
- └─ L3 → 双人复核 → 带时效令牌 → 执行器 → ...
- └─ L4 → 拒绝 (解释原因 + 留痕)

→ 输出到对话框

→ 有价值的业务知识 → 用户确认 → 候选池 → 责任人审核 → 知识库 (带版本/来源)

→ 查询/调度结果 → 由 renderer (格式化 skill) 规范化展示 (依赖图 / 结果表 / SQL / 执行计划)

→ 全链路审计 (问题、skill 调用、判定依据、回执、确认人)

二次查询: Harness → 防火墙 → 知识库 (优先本地知识, 命中即不再打扰模型) → 重复

初始化 (可选, 一次性): 调度器已有任务 → 批量解析 → 报告 + 候选知识 (默认不落库) → 人工抽检 → 入库

6. 待敲定 (10 条, 附我的建议)

#	问题	我的建议
1	LLM 为主后, "无 key 也能跑"还要不要保留?	保留降级模式 (只读能力走规则), 但 默认走模型 ; 降级时界面必须显式标注
2	事实校验放在哪一层?	harness 出口统一校验 (回执白名单), 不在每个 skill 里各做一遍
3	防火墙是独立服务还是进程内模块?	P1 进程内模块 + 策略表 (够用、好测); 多租户/多环境后再独立部署
4	L2/L3 的"确认人"是谁?	L2: 提问本人; L3: 表责任人 (知识库里已有责任人字段) + 提问者, 双人
5	审批令牌怎么防复用?	一次性 + 绑定动作指纹 (表名+语句 hash) + 5 分钟时效
6	知识回写的审核链?	候选池 → 责任人审核 → 入库 (版本化), 不允许模型直接写正式口径
7	会话记忆与知识库边界?	会话记忆=临时 (会话内); 知识库=长期 (跨会话、需审核)。 不把会话内容自动灌进知识库
8	多环境 (开发/测试/生产) 怎么隔离?	连接与权限按环境分档, 生产默认只读 ; 跨环境动作视为 L3
9	前端结果渲染怎么可插拔?	skill 声明 <code>renderer</code> , 前端按名注册视图组件 (先做 table/graph/sql/diff 四种)
10	初始化批量解析的范围?	先做一个项目、 只出报告不落库 , 人工抽检后再全量; 落库走候选池

7. 我们的协作方式（按你的决定调整）

你说得对——**这个对话不再做代码开发**（我已经把开发交给协作者）。我在这里的角色改为：

我做什么	具体交付
原型	交互原型、界面变体、可点的演示版（零构建，直接给人看）
功能点	把架构拆成可认领的工作项（含验收方式、依赖、规模）
任务拆解	每个工作项拆到"一个 PR 能合并"的粒度，写清接口与边界
合并代码	审协作者的 PR：跑门禁、验验收路径、检查分层与契约，通过后合并
验收	按验收清单逐条验（真跑、真数据、有截图/输出），不合格打回

配套流水线（沿用已有的「协作与分工机制」）：`Issue` → 分支 → PR → 我审（门禁+验收）→ 合并 → 演示资产更新。

你只需要在**验收点**拍板，以及回答上面那 10 条待敲定。

8. 决策已定（2026-09-25 用户逐条确认）

#	问题	决定	落地影响
1	是否保留"无模型降级"	不保留 （暂时）	无决策模型 = 平台不可用：必须 明确报错 + 留痕 ，不能静默失败；测试用 mock 模型
2	事实校验放哪	harness 出口统一校验 （回执白名单）	不在每个 skill 里各做一遍；只允许用回执中的数字/标识符组答案
3	防火墙形态	独立部署的服务	服务间签名调用；策略与判定可单独扩容与审计
4	审批复杂度	只保留最高档需审批 + 上一级确认 ，暂不做审批模块	风险判定简化为三档：自动通过 / 需上级审批 / 禁止
5	令牌机制	一次性 + 绑动作指纹 + 5 分钟时效	复用他人令牌不可行
6	知识回写审核链	候选池 → 责任人审核 → 版本化入库	禁止模型直写正式口径；冲突必须仲裁
7	会话记忆 vs 知识库	会话记忆临时；知识库长期且需审核	不把会话内容自动灌进知识库
8	跨环境动作	视为"需审批"档，本人确认一次	跨环境不再单独分级，与生产写同档
9	前端渲染可插拔	按 skill 声明的 renderer 注册视图	先做 table / graph / sql / diff 四种
10	初始化批量解析	先一个项目 + 只出报告不落库 + 人工抽检	合格后才走候选池入库

确认版图：`docs/platform-vision/diagrams/harness-architecture-v3.html`（v2 为讨论稿，v3 为确认稿）。

9. 技术选型与契约 (2026-09-25 已定)

9.1 Skill 契约 (四条原则, 用户已确认)

1. **声明与实现分开**: 声明文件 (YAML) + 代码实现; 加一个 skill = 加一个目录, **不改 harness 一行代码**。
2. **一个 skill = 一个动作**: 流程编排留在 harness; 确需组合的能力显式标 `composite: true`, 不许把编排藏进 skill。
3. **skill 只申报风险, 防火墙最终裁决**: 即使有人把 `drop table` 声明成只读, 也按策略表拒绝——**安全不靠自觉**。
4. **新增 skill 必须走 PR + 契约校验**: 缺字段、声明与实现不符 → 门禁红, 不许合并。

字段集见 §3。两条安全细节同批确认: **要执行的 SQL 必须由生成类 skill 产出并先过校验** (生成与执行分离); **每个 skill 返回必须带回执** (来源端点/耗时/影响行数), 供 harness 出口校验。

9.2 选型

#	选型	决定	备注 / 风险
A	决策模型	DeepSeek API (先走 API)	业余开发、暂无正式环境; 统一走 OpenAI 兼容协议 + 薄模型网关 (换模型/限流/记账不改业务)
B	harness 底座	采用 DeepSeek Harness (dsh)	MIT TypeScript + Cordis 插件框架 官方定位 "Everything is a Plugin" v0.1 开发者预览版 (2026-08 开源)
C	skill 暴露	MCP	skill 声明叠加在 MCP 之上: MCP 管"怎么调", 契约管"能不能调、怎么展示"

采用 dsh 的边界 (必须守住): 防火墙、出口事实校验、审计落库这三件留在**自有侧**, 不塞进 dsh。

风险与对策: dsh 处于预览期、API 可能破坏性变更 → ① 锁定版本, 不追 latest; ② skill 契约锁在 **MCP** 层, 外壳可替换而 skill 资产不动。

9.3 知识库: 双通道 (用户已确认)

通道	存什么	技术	角色边界
① 结构化通道 (核心)	口径/公式、表、字段、术语、规则、血缘	既有知识库 + 实体锚定 + 精确匹配优先 + 强制来源 (脚本 + 行号)	唯一允许产出结论数字的通道
② 文档通道 (补充)	脚本注释、需求文档、SOP、会议纪要	WeKnora (腾讯开源, MIT, v0.8.0: BM25 + 向量 + 重排 + GraphRAG; 官方 MCP server 29 工具)	只提供背景说明, 不产出结论

为什么不能只靠文档 RAG: v1 实测——一次检索返回 72 条命中, 若不按"表名/字段名实体"过滤, 会把「产量怎么来的」答成"上游透传" (错的), 因为真实公式在 **dwd** 层而非 **ads** 层。

知识提炼流水线 (唯一没做完的一段):

脚本/任务流 → 解析血缘 → LLM 初筛候选口径 → 规则校验 (必须有公式 + 依赖字段 + 来源脚本)
→ 候选池 → 责任人审核 → 版本化入库 (可回滚)

质量门禁: 没有来源脚本的口径一律不许入库; 同字段两条公式冲突必须仲裁, 不允许并存。

9.4 本批决定之后的待交付

交付物	内容	状态
《Skill 契约规范 v1》	契约字段规范 + skill 模板 + 标杆示例 （把"解析血缘"改写成标准 skill）	待做
《harness 选型建议 v1》	dsh 采用方式、版本锁定策略、风险清单、自有三层的接口	待做
《知识库选型与应用 v1》	双通道设计、提炼流水线、质量门禁、WeKnora 接入方式（MCP）	待做
v2 架构工作项拆解	按本图拆到"一个 PR 能合并"的粒度（接口/边界/验收方式）	待做